

Online Algorithms Fundamentals

1 Core Idea

Online Algorithm

An **online algorithm** receives input requests one by one. After each request arrives, it must make a decision before seeing future requests.

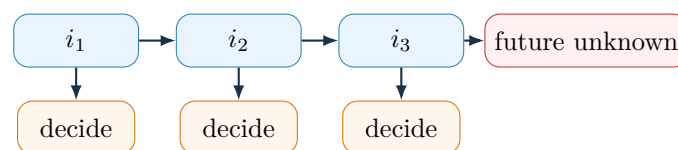
Offline Algorithm

An **offline algorithm** sees the whole input sequence in advance. It can use future information to make the best possible decisions.

Main Difference

Online algorithms make decisions with incomplete information. Offline algorithms make decisions with complete information.

2 Decision Timeline



After request i_j , the algorithm knows only

$$i_1, i_2, \dots, i_j.$$

It does not know

$$i_{j+1}, i_{j+2}, \dots$$

3 Examples of Online Problems

Problem	Online decision	Why future matters
Ski rental	Rent today or buy skis.	The season length is unknown.
Caching	Choose which page to evict.	Future page requests are unknown.
Scheduling	Assign a job to a machine.	Future jobs may be larger or smaller.
Routing	Move a server or route a request.	Future request locations are unknown.

4 Cost Notation

Let I be one complete input sequence.

$C_A(I)$ = cost paid by online algorithm A on I .

$\text{OPT}(I)$ = minimum possible offline cost on the same I .

Fair Comparison

Both algorithms are tested on the same input I . The online algorithm sees I gradually, while OPT sees all of I before deciding.

5 Competitive Ratio

Cost-Minimization Ratio

For a minimization problem,

$$\text{competitive ratio of } A = \max_I \frac{C_A(I)}{\text{OPT}(I)}.$$

The maximum is over all possible input sequences. If

$$C_A(I) \leq r \cdot \text{OPT}(I)$$

for every input I , then A is called **r -competitive**.

How to Read r

$r = 1$ means optimal, $r = 2$ means at most twice the offline optimum.

6 Additive Constant

Sometimes the definition allows a fixed constant b :

$$C_A(I) \leq r \cdot \text{OPT}(I) + b.$$

The constant b handles very small inputs where a fixed startup cost can distort the ratio. For most introductory calculations, use the simpler ratio

$$\frac{C_A(I)}{\text{OPT}(I)}.$$

7 Worked Example

Problem

For three input sequences, an online algorithm A and the offline optimum have these costs:

I	I_1	I_2	I_3
$C_A(I)$	12	15	10
$\text{OPT}(I)$	8	10	5

Find the worst observed competitive ratio.

Solution

Compute the ratio for each input:

$$\frac{C_A(I_1)}{\text{OPT}(I_1)} = \frac{12}{8} = \frac{3}{2}.$$

$$\frac{C_A(I_2)}{\text{OPT}(I_2)} = \frac{15}{10} = \frac{3}{2}.$$

$$\frac{C_A(I_3)}{\text{OPT}(I_3)} = \frac{10}{5} = 2.$$

The worst observed ratio is the largest one:

$$\max \left\{ \frac{3}{2}, \frac{3}{2}, 2 \right\} = 2.$$

Answer

Worst observed ratio = 2.

8 How to Prove an Algorithm Is Competitive

Proof Template

To prove that an online algorithm A is r -competitive, show that every input I satisfies

$$C_A(I) \leq r \cdot \text{OPT}(I).$$

A typical proof has three parts:

1. Describe the cost paid by the online algorithm.
2. Give a lower bound on $\text{OPT}(I)$.
3. Compare the two expressions and show the ratio is at most r .

Important

Do not compare the online algorithm to a weak offline strategy. Compare it to OPT, the best possible offline solution.

9 Minimization vs Maximization

For cost problems, smaller is better, so use

$$\frac{C_A(I)}{\text{OPT}(I)}.$$

For profit problems, larger is better, so the ratio is usually written in the opposite direction:

$$\frac{\text{OPT}(I)}{C_A(I)}.$$

Use the Correct Direction

Cost problem:

$$C_A(I) \leq r \text{OPT}(I).$$

Profit problem:

$$\text{OPT}(I) \leq r C_A(I).$$

10 Deterministic and Randomized Online Algorithms

Type	Meaning
Deterministic online algorithm	Makes the same decision whenever it sees the same history.
Randomized online algorithm	Uses random choices while making online decisions.

For randomized algorithms, we usually compare the **expected cost** to the offline optimum:

$$\mathbb{E}[C_A(I)] \leq r \cdot \text{OPT}(I).$$

11 Adversarial Inputs

Competitive analysis is worst-case. This means the input sequence may be chosen to make the online algorithm look bad.

Why Worst Case?

Online algorithms cannot rely on the future being friendly. Competitive analysis asks how well the algorithm performs even on difficult input sequences.

12 Review Box

Core Facts

Online algorithm: sees requests one at a time.

Offline optimum: sees the whole input first.

$C_A(I)$ = online cost on input I .

$\text{OPT}(I)$ = best offline cost on I .

For minimization: A is r -competitive if $C_A(I) \leq r \text{OPT}(I)$.

Competitive ratio measures worst-case loss from not knowing the future.

Ski Rental Problem

1 Problem

Ski Rental

A skier does not know how many days they will ski. Each day, before knowing the future, the skier must choose one of two actions:

rent for one day or buy once and use forever.

Use the values

$$R = 500, \quad B = 1500,$$

where R is the rental cost per day and B is the buying cost.

Let d be the actual number of ski days.

2 Offline Optimum

If d were known in advance, the best offline strategy would be:

$$\text{OPT}(d) = \min(Rd, B).$$

For our values,

$$\text{OPT}(d) = \min(500d, 1500).$$

Meaning

The offline optimum either rents for all d days or buys immediately. It chooses whichever is cheaper after seeing the whole future.

3 Break-Even Day

The break-even point is the number of rental days whose total cost equals buying:

$$\frac{B}{R} = \frac{1500}{500} = 3.$$

So:

$$3 \text{ rental days cost } 3 \cdot 500 = 1500,$$

which is exactly the cost of buying.



4 Bad Strategy: Always Rent

Cost

$$C_{\text{rent}}(d) = 500d.$$

If $d \leq 3$, this is fine. But if d becomes large, the offline optimum buys once and pays only 1500. For example, if $d = 10$,

$$C_{\text{rent}}(10) = 5000, \quad \text{OPT}(10) = 1500.$$

The ratio is

$$\frac{5000}{1500} = \frac{10}{3}.$$

As d grows,

$$\frac{500d}{1500} = \frac{d}{3}$$

also grows without bound.

Conclusion

Always renting is not competitive because its competitive ratio is unbounded.

5 Bad Strategy: Buy Immediately

Cost

$$C_{\text{buy-now}}(d) = 1500.$$

This is good when the skier skis many days, but bad when the skier stops early.

If $d = 1$, then

$$C_{\text{buy-now}}(1) = 1500, \quad \text{OPT}(1) = 500.$$

So

$$\frac{C_{\text{buy-now}}(1)}{\text{OPT}(1)} = \frac{1500}{500} = 3.$$

Conclusion

Buying immediately is 3-competitive.

6 Good Strategy: Rent Twice, Then Buy

Since the break-even point is day 3, use this deterministic online strategy:

Threshold Strategy

Rent on days 1 and 2. If skiing continues to day 3, buy at the start of day 3.

The online cost is

$$C_A(d) = \begin{cases} 500d, & d \leq 2, \\ 2500, & d \geq 3. \end{cases}$$

The value 2500 comes from

$$2 \cdot 500 + 1500 = 2500.$$

7 Why the Ratio Is $\frac{5}{3}$

We prove

$$C_A(d) \leq \frac{5}{3} \text{OPT}(d)$$

for every possible number of days d .

Case 1: $d \leq 2$

The algorithm rents for every ski day:

$$C_A(d) = 500d.$$

The offline optimum also rents:

$$\text{OPT}(d) = 500d.$$

Thus

$$\frac{C_A(d)}{\text{OPT}(d)} = 1.$$

Since

$$1 \leq \frac{5}{3},$$

the bound holds.

Case 2: $d \geq 3$

The algorithm rents twice and then buys:

$$C_A(d) = 2500.$$

The offline optimum buys, because renting 3 or more days costs at least 1500:

$$\text{OPT}(d) = 1500.$$

Therefore

$$\frac{C_A(d)}{\text{OPT}(d)} = \frac{2500}{1500} = \frac{5}{3}.$$

Competitive Ratio

Rent twice, then buy is $\frac{5}{3}$ -competitive.

8 Cost Table

Days d	Online $C_A(d)$	Offline $\text{OPT}(d)$	Ratio
1	500	500	1
2	1000	1000	1
3	2500	1500	$\frac{5}{3}$
4	2500	1500	$\frac{5}{3}$
10	2500	1500	$\frac{5}{3}$

The worst ratio starts when the skier reaches day 3.

9 General Rule

Let:

R = daily rental cost, B = buying cost.

If B/R is an integer, then the standard deterministic threshold strategy is:

rent for $\frac{B}{R} - 1$ days, then buy on the next day.

Its competitive ratio is

$$2 - \frac{R}{B}.$$

With the given values,

$$2 - \frac{R}{B} = 2 - \frac{500}{1500} = 2 - \frac{1}{3} = \frac{5}{3}.$$

10 Strategy Comparison

Strategy	Worst ratio	Problem
Always rent	Unbounded	Very expensive if the season is long.
Buy immediately	3	Too expensive if the skier stops after one day.
Rent twice, then buy	$\frac{5}{3}$	Balances early stopping and long seasons.

11 Review Box

Core Facts

$$\text{OPT}(d) = \min(Rd, B).$$

$$\text{Break-even day} = B/R.$$

For $R = 500$, $B = 1500$, break-even day = 3.

Best simple deterministic strategy: rent days 1, 2, buy on day 3.

$$\text{Competitive ratio} = \frac{2500}{1500} = \frac{5}{3}.$$

Linear List Search

1 Problem

Linear List Search

We have a list of n items. A request asks for one item. If the item is in position k , then accessing it costs k .

The input is a request sequence

$$\sigma = (x_1, x_2, \dots, x_m).$$

After serving a request, the algorithm may reorder the list. The goal is to minimize the total access cost.

Online Difficulty

The algorithm sees only the current request. It does not know which items will be requested later.

2 Example

Suppose the list is

$$(1, 2, 3, 4, 5).$$

If the request is 4, then the access cost is

$$4.$$

After accessing 4, an online algorithm may move it closer to the front, hoping it will be requested again.

3 Strategy 1: Move Forward by 2

Move Forward by 2

After accessing an item, move it two positions closer to the front of the list.

This strategy looks reasonable, but it can perform very badly.

Bad Request Sequence

Initial list:

$$(1, 2, 3, \dots, n).$$

Request the last item repeatedly:

$$\sigma = (n, n, n, \dots, n).$$

The first access costs n . After that, the item moves forward by only two positions. So the first few costs are

$$n + (n - 2) + (n - 4) + \dots$$

For about $n/2$ requests, the accessed item is still moving toward the front. The total cost is therefore quadratic:

$$n + (n - 2) + (n - 4) + \dots = cn^2$$

for some positive constant c . The exact constant is not important. What matters is that the cost grows like n^2 .

The offline optimum knows that n will be requested repeatedly. It can pay n once, move n to the front, and then pay 1 for later repeated requests:

$$n + 1 + 1 + \dots + 1 = 2n - 1 < 2n.$$

Competitive Ratio

The ratio is at least

$$\frac{\text{online cost}}{\text{offline cost}} > \frac{cn^2}{2n} = \frac{c}{2}n.$$

As

$$n \rightarrow \infty,$$

we get

$$\frac{c}{2}n \rightarrow \infty.$$

Conclusion

Move-forward-by-2 is not competitive. Its competitive ratio becomes unbounded as n becomes large.

4 Strategy 2: Move To Front

Move To Front

The **Move-To-Front** strategy, abbreviated MTF, moves the requested item to the front immediately after accessing it.

For example, if the list is

$$(1, 2, 3, 4, 5)$$

and the request is 4, then the cost is 4, and the new list becomes

$$(4, 1, 2, 3, 5).$$

Main Theorem

MTF is 2-competitive.

5 Why MTF Works

MTF is good because it reacts strongly to repeated requests. If an item is requested once, it moves to the front, so another request for the same item becomes cheap.

The proof compares two algorithms:

Algorithm	Meaning
MTF list	The list maintained by the online algorithm.
OPT list	The list maintained by the optimal offline algorithm.

At the beginning, assume both lists are the same:

$$\text{OPT} : (1, 2, 3, \dots, n), \quad \text{MTF} : (1, 2, 3, \dots, n).$$

At request j , suppose the requested item is i . If i is at position k in the MTF list, then MTF pays cost k .

6 Inversions Between Two Lists

Inversion

An **inversion** is a pair of items whose relative order is different in the MTF list and in the OPT list.

Example:

$$\begin{aligned} \text{OPT} : & \quad \dots a \dots b \dots \\ \text{MTF} : & \quad \dots b \dots a \dots \end{aligned}$$

The pair (a, b) is an inversion because the two lists disagree about which item comes first.

Let

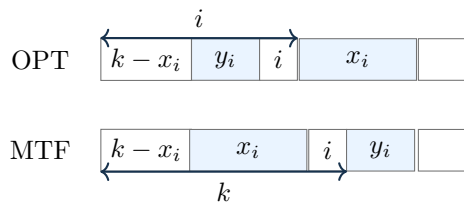
$$\text{INV}(j)$$

be the number of inversions before request j .

7 The x_i and y_i Groups

For the requested item i , define:

Symbol	Meaning
x_i	Number of items after i in OPT but before i in MTF.
y_i	Number of items before i in OPT but after i in MTF.



The x_i items are before i in MTF, so MTF pays for passing them. In OPT, those same items are after i .

8 Amortized Cost

Suppose MTF accesses item x at position k . Its real cost is k .

Instead of analyzing only the real cost, define the amortized cost:

$$T(j) = k + \text{INV}(j+1) - \text{INV}(j).$$

This means:

- pay the real access cost k ;
- add the change in inversion count.

Why Use Inversions?

The inversion count measures how different MTF's list is from OPT's list. If MTF moves an item to the front and becomes closer to OPT, the inversion count decreases and helps pay for the move.

9 One-Step MTF Bound

When MTF moves item i to the front, the inversion count changes.

- x_i old inversions disappear.
- At most $k - x_i$ new inversions appear.
- Let $\alpha \geq 0$ be the number of additional inversions removed by OPT's own reordering.

Thus the amortized cost satisfies

$$T(j) = k + ((k - x_i) + y_i - \alpha) - (x_i + y_i).$$

Simplify:

$$T(j) = 2k - 2x_i - \alpha.$$

Since $\alpha \geq 0$,

$$T(j) \leq 2k - 2x_i.$$

So

$$T(j) \leq 2(k - x_i).$$

From the diagram, OPT's position of item i is at least $k - x_i$. Therefore

$$k - x_i \leq \text{OPT cost for request } j.$$

Hence

$$T(j) \leq 2 \cdot \text{OPT cost for request } j.$$

10 Telescoping the Amortized Costs

For each request:

$$\begin{aligned} T(1) &= k_1 + \text{INV}(2) - \text{INV}(1), \\ T(2) &= k_2 + \text{INV}(3) - \text{INV}(2), \\ &\dots \\ T(m) &= k_m + \text{INV}(m+1) - \text{INV}(m). \end{aligned}$$

Now sum all requests:

$$\sum_{j=1}^m T(j) \leq 2 \text{OPT}(\sigma).$$

But the amortized costs telescope:

$$\sum_{j=1}^m T(j) = \sum_{j=1}^m k_j + \text{INV}(m+1) - \text{INV}(1).$$

Here k_j is the real cost paid by MTF on request j . Therefore the total real cost of MTF is

$$C_{\text{MTF}}(\sigma) = \sum_{j=1}^m k_j.$$

The initial lists are the same, so there are no inversions at the beginning:

$$\text{INV}(1) = 0.$$

Also, an inversion count can never be negative:

$$\text{INV}(m+1) \geq 0.$$

So the telescoping equation becomes

$$\sum_{j=1}^m T(j) = \sum_{j=1}^m k_j + \text{INV}(m+1).$$

The right side is the real MTF cost plus a nonnegative extra term. Therefore

$$\sum_{j=1}^m k_j \leq \sum_{j=1}^m T(j).$$

Earlier we proved each request satisfies

$$T(j) \leq 2 \cdot \text{OPT cost on request } j.$$

Adding this inequality over all requests gives

$$\sum_{j=1}^m T(j) \leq 2 \text{OPT}(\sigma).$$

Combining the two bounds:

$$C_{\text{MTF}}(\sigma) = \sum_{j=1}^m k_j \leq \sum_{j=1}^m T(j) \leq 2 \text{OPT}(\sigma).$$

Hence

$$C_{\text{MTF}}(\sigma) \leq 2 \text{OPT}(\sigma).$$

Result

$$C_{\text{MTF}}(\sigma) \leq 2 \text{OPT}(\sigma).$$

Therefore, MTF is 2-competitive.

11 Why No Online Algorithm Can Do Much Better

There is a lower bound for linear list search:

Lower Bound

No deterministic online algorithm for linear list search has competitive ratio better than

$$2 - \frac{2}{n+1}.$$

This means MTF's ratio 2 is almost optimal when n is large.

12 Lower Bound Idea

Consider any online algorithm A . An adversary can choose requests maliciously.

At each step, the adversary requests the item currently at the last position in A 's list. Therefore, A pays

$$n$$

on every request. For m requests,

$$C_A(\sigma) = mn.$$

Now compare this with an offline algorithm. The offline algorithm knows the whole request sequence

$$\sigma = x_1, x_2, \dots, x_m$$

in advance. We do not need to know the exact optimal offline strategy. It is enough to show that there exists one fixed list order whose cost is small, because OPT can only be better than that fixed order.

Best Fixed Permutation Argument

Generate all possible fixed permutations of the n items:

$$\pi_1, \pi_2, \dots, \pi_{n!}.$$

For the given request sequence σ , let $O(\pi_i)$ be the total search cost if we use permutation π_i and never change it. The best fixed permutation has cost

$$\min_i O(\pi_i).$$

Since a minimum is always at most the average,

$$\min_i O(\pi_i) \leq \frac{O(\pi_1) + O(\pi_2) + \dots + O(\pi_{n!})}{n!}.$$

Now compute that average. Look at one request x_j . Across all $n!$ permutations, the requested item appears:

$(n-1)!$ times in position 1,

$(n-1)!$ times in position 2,

and so on up to position n . Therefore the total cost of searching this one requested item over all permutations is

$$(n-1)!(1 + 2 + \cdots + n).$$

Using

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

we get

$$(n-1)! \frac{n(n+1)}{2} = \frac{n!(n+1)}{2}.$$

So the average cost of this one request over all $n!$ permutations is

$$\frac{n!(n+1)/2}{n!} = \frac{n+1}{2}.$$

There are m requests, so the average total cost of a fixed permutation is

$$\frac{m(n+1)}{2}.$$

Hence at least one fixed permutation has cost at most

$$\frac{m(n+1)}{2}.$$

Since OPT is the best offline algorithm,

$$\text{OPT}(\sigma) \leq \min_i O(\pi_i) \leq \frac{m(n+1)}{2}.$$

Thus

$$\frac{C_A(\sigma)}{\text{OPT}(\sigma)} \geq \frac{mn}{m(n+1)/2} = \frac{2n}{n+1} = 2 - \frac{2}{n+1}.$$

Meaning

For large n ,

$$2 - \frac{2}{n+1} \approx 2.$$

So no online algorithm can guarantee a ratio significantly below 2, and MTF is essentially best possible.

13 Strategy Comparison

Strategy	Competitive behavior	Reason
Move forward by 2	Not competitive	Repeated requests to a deep item remain expensive for many steps.
Move to front	2-competitive	Repeatedly requested items become cheap immediately.
Best possible online ratio	At least $2 - \frac{2}{n+1}$	Adversary can always request the current last item.

14 Review Box

Core Facts

Access cost = position of requested item in the list.

Move-forward-by-2 is not competitive.

MTF moves each requested item to the front.

MTF is 2-competitive.

No deterministic online algorithm can beat $2 - \frac{2}{n+1}$.

For large n , this lower bound is close to 2.